

A Multi-Agent and MCP-Based Customer Churn Retention Solution for Automotive 4S Dealerships: Design, Observability, and Evaluation Practices

Author: Qin Lu

Date: 24th Sep 2026

Project: Multi-Agent Customer Retention Workflow

Table of Contents

1. Solution Design
2. Agent Architecture
3. Observability Strategy
4. Evaluation Approach
5. Workflow & Deployment

1. Solution Design

1.1 Background and Objectives

This solution targets a customer churn prevention and retention use case for automotive dealerships within the Vehicle Group. The objective is to build a Multi-Agent workflow that starts from customer data and autonomously performs end-to-end churn risk assessment, retention strategy decisioning, personalised email generation, and subsequent actions through MCP-enabled tools.

1.2 Two Candidate Designs and Selection Rationale

Design 1: TurboApply General-Purpose Recommendation Framework

This design is derived from my self-developed job recommendation tool, TurboApply. The existing capabilities are already integrated into TurboApply, with the longer-term goal of evolving the framework into a general-purpose market intelligence and strategic recommendation tool that can be applied across different users and organisations. The proposed workflow is:

- (1) Input: Target company name.
- (2) Agent 1 — Company Profiling: Generate a structured company profile, including industry, business description, and relevant tags.
- (3) Agent 2 — Similarity & Competitive Intelligence: Use the company profile to identify the Top-K most similar companies. For job seekers, this can provide a Top-10 target-company shortlist; for enterprise customers, it can identify the top competitor or most relevant competitive peer.
- (4) MCP Integration: Connect to MCP-enabled tools to retrieve the company's latest news and relevant market intelligence.

This architecture provides a reusable foundation for company intelligence, market analysis, competitive discovery, and recommendation workflows.

Design 2: Multi-Agent Customer Retention Workflow

This design is derived from my previous project, [Hybrid-ML-LLM-Retrieval-System-for-Customer-Churn-Decisioning](#). The original system primarily used machine learning for churn prediction, with LLMs providing supporting decision intelligence. Although it did not incorporate Agentic AI, the underlying business scenario is naturally suited to a Multi-Agent architecture.

The proposed workflow is:

- (1) Input: Customer attributes and behavioural features relevant to churn assessment.
- (2) Agent 1 — Churn Risk Assessment: Generate a customer churn risk rating based on the available customer data and ML-derived signals.
- (3) Agent 2 — Retention Strategy: Recommend personalised promotional or retention actions based on the customer's risk profile and relevant characteristics.

- (4) Agent 3 — Personalised Communication: Generate a tailored promotional email aligned with the recommended retention strategy.
- (5) Agent 4 — Action Execution: Use the Outlook MCP to schedule and send the personalised email.

Selection Rationale

Design 1 offers greater potential in terms of innovation, extensibility, and general-purpose applicability, but would require a longer implementation cycle to achieve a robust end-to-end solution.

Design 2 provides a more mature starting point, enables direct reuse of existing ML assets, and maps naturally to the customer retention scenario. More importantly, it allows the complete data → decision → communication → action workflow to be implemented and validated end-to-end within the available event timeframe.

Therefore, Design 2 — the Multi-Agent Customer Retention Workflow — was selected as the implementation approach.

2. Agent Architecture

2.1 Agent Roles and Responsibilities

The solution consists of four specialised agents, each responsible for a clearly bounded stage of the customer retention workflow. The architecture follows a separation-of-concerns principle: risk assessment, retention decisioning, content generation, and tool execution are handled independently.

Table 1. Agent Roles and Responsibilities

Agent	Responsibility	Input	Output
churn-triage-agent	Customer churn triage and risk assessment	Four customer churn indicators	VIN, vehicle summary, churn risk level, flagged metrics
churn-decision-agent	Retention promotion decisioning	Triage output	A single recommended retention action and rationale
email-content-agent	Personalised email generation	Decision output	Email subject and body
auto-send-agent (MCP)	Tool invocation and action execution	Generated email content	MCP tool execution result

(1) Churn Triage Agent — Key Design

The churn-triage-agent evaluates four key indicators:

- customer_value_score — higher values indicate stronger customer value.
- churn_risk_score — lower values indicate lower churn risk.
- rfm_loyalty_score — higher values indicate stronger customer loyalty.
- vehicle_usage_lifecycle_score — lower values indicate a lower likelihood of vehicle lifecycle-related churn.

Each metric has a predefined acceptable range [min, max]. A metric is flagged when it falls outside its acceptable range. The deviation represents the magnitude by which the metric exceeds the relevant threshold.

The overall churn risk is then categorised based on the number of flagged indicators:

Risk Level	Condition
 High	3 or more flagged indicators
 Medium	1–2 flagged indicators
 Low	No flagged indicators

This provides a deterministic and explainable triage layer before any LLM-based retention decision is made.

(2) Churn Decision Agent — Key Design

The churn-decision-agent is deliberately constrained to recommend exactly one retention promotion:

- extended_warranty_insurance — recommended when the warranty has expired or is approaching expiry and vehicle mileage is below 80,000 km.

- `new_car_trade_in` — recommended when vehicle mileage is $\geq 80,000$ km and vehicle age is exactly 4 years.
- `maintenance_package` — used as the default recommendation when the above conditions are not met.

If multiple rules are triggered, the agent selects the promotion that addresses the strongest observed churn signal.

The output follows a fixed structure:

RECOMMENDED ACTION: ...

REASONING: ...

This constraint prevents the agent from producing multiple competing recommendations and makes the downstream workflow deterministic and auditable.

(3) Email Content Agent — Key Design

The email-content-agent is responsible solely for translating the approved retention decision into customer-facing communication.

The agent must:

- Generate content strictly based on the recommended promotion without re-evaluating or overriding the decision.
- Personalise the message using relevant customer and vehicle attributes, including vehicle age, mileage, warranty status, and churn signals.
- Keep the email within 200 words.
- Avoid fabricating prices, discounts, promotional deadlines, or other unsupported commercial information.
- Follow the fixed output format:

SUBJECT: ...

EMAIL: ...

This separation ensures that the communication layer does not introduce new business decisions after the retention strategy has already been approved.

(4) Auto-Send Agent — MCP Integration

The original design planned to use an Outlook MCP integration to automatically send the generated email. Due to the limited implementation timeframe, this was replaced by a generic MCP-enabled agent.

The agent was configured with the following general instruction:

"You are a helpful agent that can use MCP tools to assist users. Use the available MCP tools to answer questions and perform tasks."

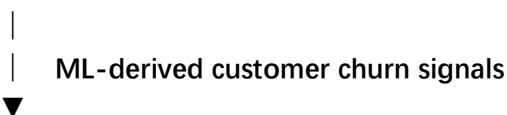
For the final implementation, the agent was connected to the Azure REST API Specifications MCP rather than an Outlook-specific email-sending MCP.

This design keeps the action-execution layer tool-agnostic, allowing the MCP integration to be replaced or extended with other tools without modifying the upstream churn assessment, decisioning, or content-generation agents.

2.2 Workflow Orchestration and Data Flow

The workflow follows a sequential pipeline in which each agent consumes the structured output of the previous stage:

Customer Data



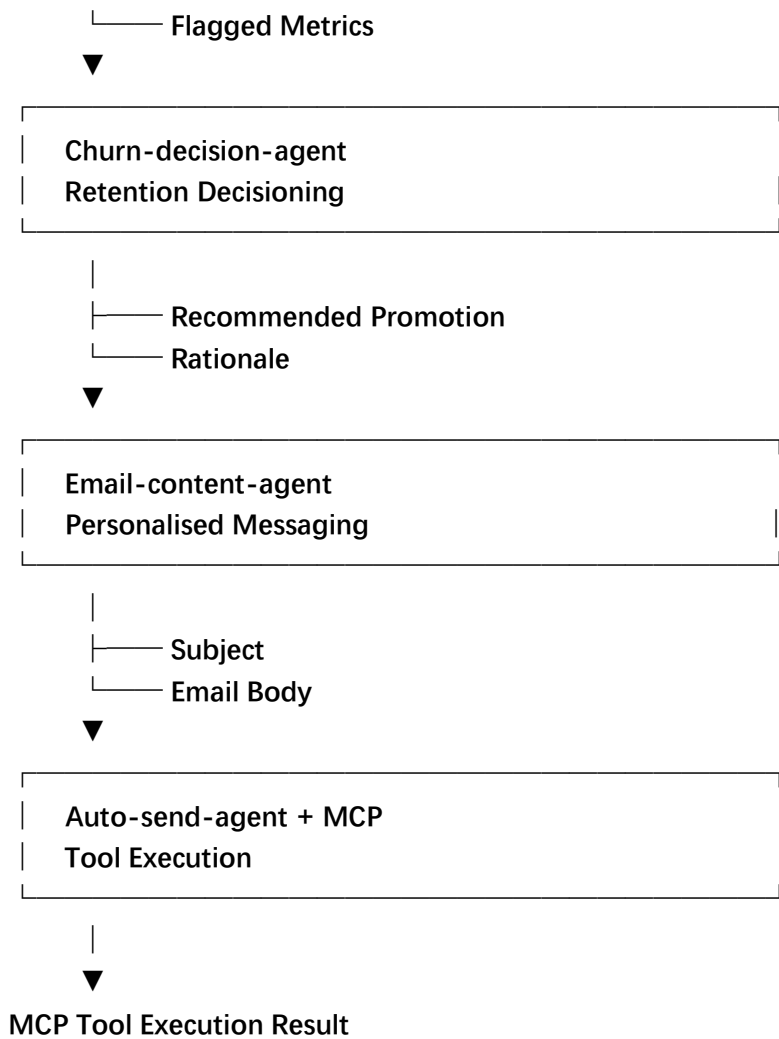
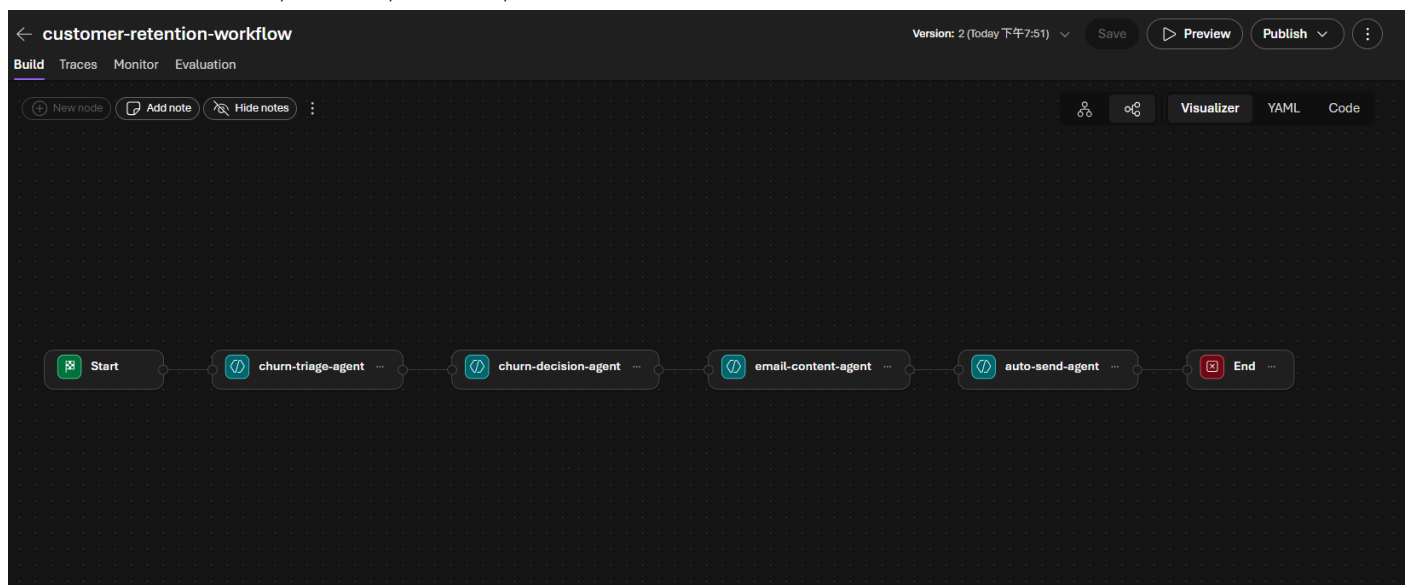


Figure 1. Four-Agent Workflow and Data Flow

The overall architecture can therefore be viewed as a decision-to-action pipeline:

Customer Data → Risk Assessment → Retention Decision → Personalised Communication → Tool Execution

Each stage has a clearly defined input/output contract, which reduces coupling between agents and makes the workflow easier to test, observe, evaluate, and extend.



2.3 MCP Integration

The MCP integration was introduced incrementally as the workflow evolved from a baseline implementation to the

full multi-agent architecture.

(1) **Baseline Implementation**

The initial baseline consisted of only two sequential agents and did not include MCP integration. This provided a minimal end-to-end implementation for validating the core workflow logic and agent-to-agent handoff before introducing additional components.

(2) **Progressive Multi-Agent and MCP Integration**

In the next stage, the remaining two agents were incorporated to complete the full four-agent workflow. The final agent was designed as the MCP-enabled action-execution layer, allowing the workflow to interact with external tools.

The original implementation plan was to integrate an Outlook MCP for automated email delivery. However, given the extremely limited implementation timeframe, the Outlook-specific integration was replaced with the Azure REST API Specifications MCP. This allowed the MCP-enabled agent and the overall workflow to be tested end-to-end without making the implementation dependent on a specific email-delivery integration.

This staged approach allowed the solution to validate the core architecture first and then progressively introduce additional agents, tool integration, and external-system interaction.

(3) **Implementation Approach: Foundry Workflow vs. Python SDK**

For workflow construction, Azure AI Foundry Workflow provided a more convenient development experience than building the orchestration entirely through the Python SDK.

In particular, the Foundry Workflow approach made it easier to:

- Define the sequence and dependencies between agents.
- Configure agent inputs and outputs visually.
- Connect agent outputs directly to downstream agents.
- Incorporate MCP-enabled tool execution into the workflow.
- Inspect and iterate on the overall orchestration more quickly.

Given the event's limited development timeframe, using Foundry Workflow as the orchestration layer reduced implementation overhead and enabled faster end-to-end validation of the multi-agent architecture.

3. Observability Strategy

3.1 Application Insights + GenAI Tracing

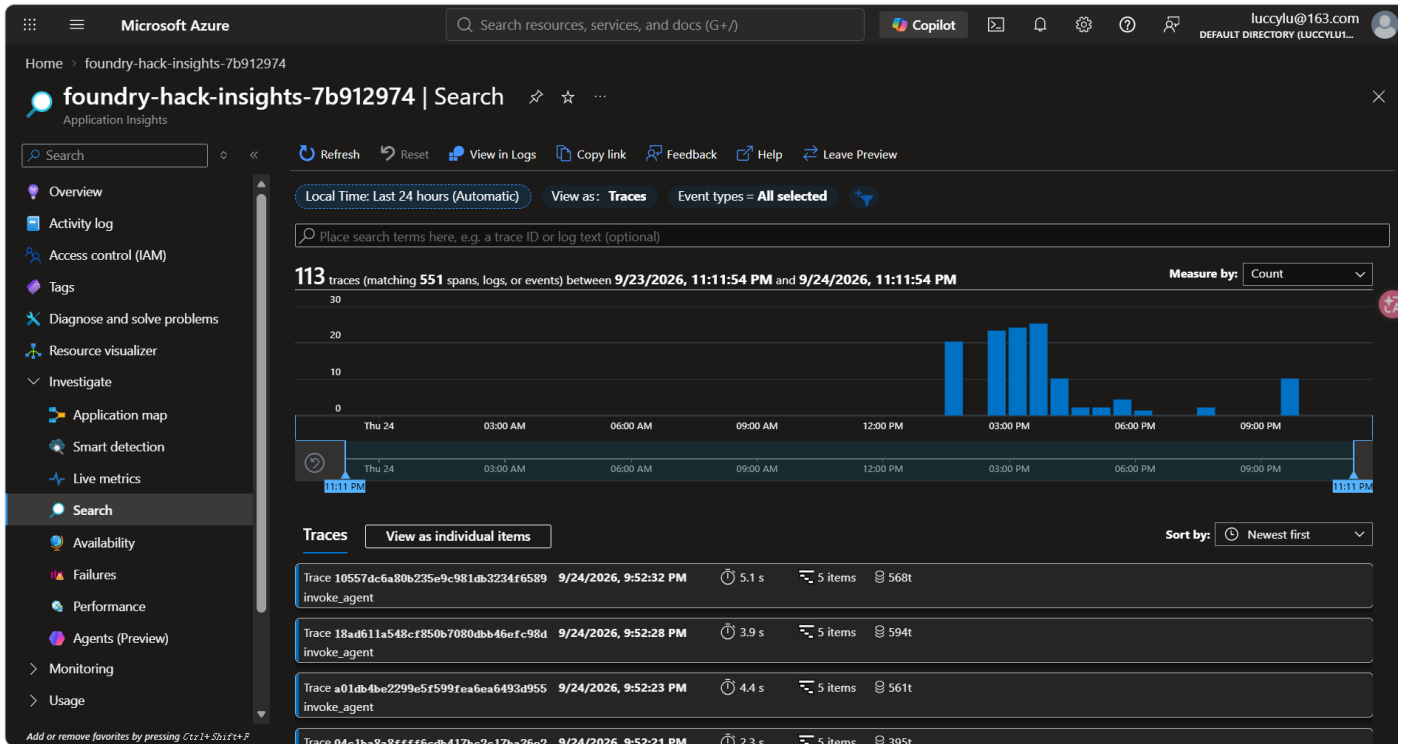
The solution uses Application Insights with GenAI tracing to provide end-to-end observability across the agentic workflow.

The tracing layer captures key operational and GenAI-specific metrics, including:

- Token usage — tracking model consumption across agent interactions.
- Latency — measuring the time spent across model calls, agent execution, and tool interactions.
- Tool execution — providing visibility into MCP and external tool invocations.
- Error tracking — capturing execution failures and supporting root-cause investigation.

This provides visibility beyond conventional application-level monitoring by exposing the behaviour and performance of the underlying GenAI workflow.

Figure2. Application Insights



3.2 Agent and Workflow Traces

In practice, the primary debugging and monitoring mechanism is the Traces and Monitor functionality provided by the Agent and Workflow runtime.

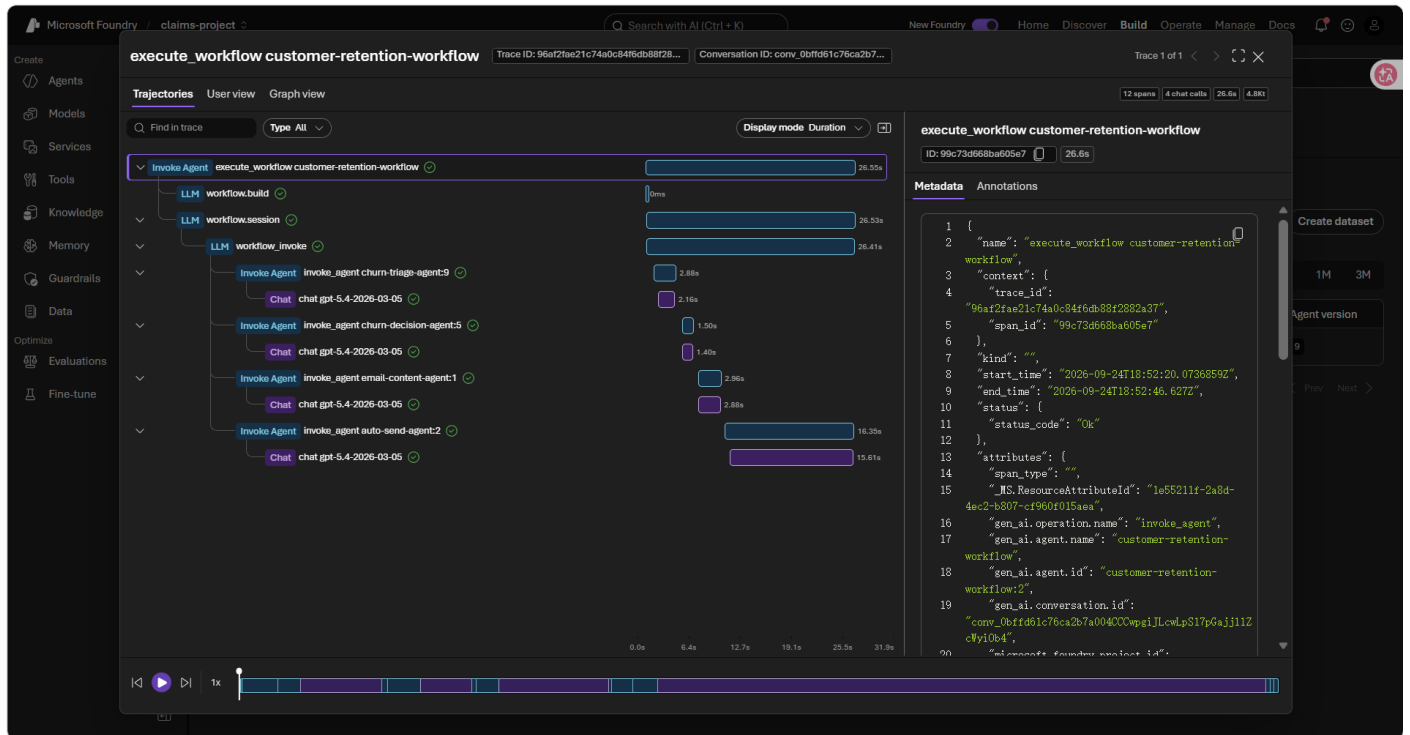
These traces provide detailed execution-level information, including:

- Agent inputs and outputs for each execution step.
- Execution metadata associated with individual agent runs.
- Error details and failure reasons.
- The progression of data through the multi-agent workflow.

This level of observability is particularly important for debugging agentic systems, where failures may originate from the model, prompt configuration, agent orchestration, tool invocation, or the data passed between stages. For example, during the V8 implementation stage, the workflow initially returned an HTTP 400 error. By inspecting the detailed execution traces, I was able to identify the specific cause of the failure rather than treating the error as a generic workflow failure.

Therefore, Traces and Monitor served as the primary source for debugging and root-cause analysis, while Application Insights provided a broader observability layer for operational and GenAI-specific monitoring.

Figure3. Traces



4. Evaluation Approach

4.1 Role of Online Evaluation

Online Evaluation is a key mechanism for assessing the quality and reliability of the agentic workflow.

Its purpose extends beyond evaluating the quality of the final agent output. It can also reveal structural issues in input data design, prompt design, agent boundaries, and workflow configuration.

This is particularly important for multi-agent systems because a seemingly strong evaluation score does not necessarily indicate that the underlying workflow is correctly designed. Evaluation results must therefore be interpreted together with the structure of the inputs, outputs, and ground-truth data.

4.2 Preventing Training Data / Label Leakage

One subtle but important issue is label leakage, where the information being used as the expected output is inadvertently included in the agent's input.

For example, suppose the intended task is to infer a churn risk level:

feature1 = 1

feature2 = 2

feature3 = risk_level

If feature3 already contains the target risk level and the LLM is then asked to predict the risk level, the model has effectively been given the answer in advance. It may simply reproduce feature3, resulting in artificially high evaluation performance.

The correct design is to provide only the features required for reasoning:

feature1 = 1

feature2 = 2

and require the agent to independently derive the risk level.

This distinction is important because evaluation quality is fundamentally dependent on input integrity. A high evaluation score obtained under label leakage does not provide meaningful evidence of the agent's decision-making capability.

4.3 Ground-Truth Dataset Design

Ground-truth data should also be designed according to the evaluation objective and metric being used.

For example, a dataset could contain multiple expected outputs:

```
{  
  "churn_risk": "high",  
  "recommended_action": "maintenance_package"  
}
```

Here, churn_risk corresponds to the expected output of Agent 1, while recommended_action corresponds to the expected output of Agent 2.

Whether these fields should be combined or separated depends on the evaluation methodology:

- For holistic metrics, such as coherence or fluency, the ground truth can be represented as a combined reference while the outputs of the individual agents are evaluated separately.
- For agent-level exact-match or structured-output evaluation, the ground truth should be separated by agent. For example, Agent 1 should be evaluated against the expected churn_risk, while Agent 2 should be evaluated independently against the expected recommended_action.

Separating ground truth at the agent level helps prevent cross-agent contamination, makes failures easier to attribute, and provides more meaningful evaluation signals for each stage of the workflow.

Overall, the evaluation strategy follows three principles:

Clean Inputs → Agent-Specific Ground Truth → Metric-Aligned Evaluation

This ensures that evaluation results measure the actual capabilities of each agent rather than artefacts introduced by the dataset or evaluation design.

4.4 Evaluation-Driven Debugging and Iteration (V1–V11)

The workflow was refined through 11 evaluation-driven iterations. Rather than making changes based on assumptions, each iteration was used to test a specific hypothesis and narrow down the root cause of evaluation failures.

Table 2. Evaluation-Driven Debugging Iterations

Version	Change	Result / Finding
V1	Removed vehicle information from the input	Evaluation failed
V2	Reintroduced vehicle information	Evaluation still failed
V3	Optimised the prompts for both agents to reduce malformed structures and make the outputs more concise	Partially successful; failures were associated with unsupported claims, but the underlying cause could not yet be isolated
V4	Corrected escape characters in the ground-truth dataset	Partially successful; the same failure pattern remained
V5	Added explicit thresholds to the prompt	Partially successful; the same issue persisted, leading to an initial hypothesis that throughput or non-parallel execution might be causing incomplete data processing
V6	Split the JSONL dataset into three smaller files for independent testing	Partially successful; the suspected throughput issue was not resolved
V7	Tested only the first VIN/customer record	Confirmed that the issue was not caused by parallelism or throughput. The query itself did not provide a sufficiently inferable reasoning path for the LLM
V8	Tested GPT-6-Luna with reasoning_effort=low	Reasoning failed; detailed traces exposed an HTTP 400 error, providing a concrete direction for further debugging

Version	Change	Result / Finding
V9	Tested GPT-6-Luna with reasoning_effort=high	Reasoning still failed, indicating that increasing reasoning effort alone did not resolve the issue
V10	Tested GPT-6-Astra with reasoning_effort=high	Reasoning succeeded
V11	Evaluated the complete dataset using the successful model configuration for both agents	Both agents achieved successful evaluation across the full dataset

Key Findings

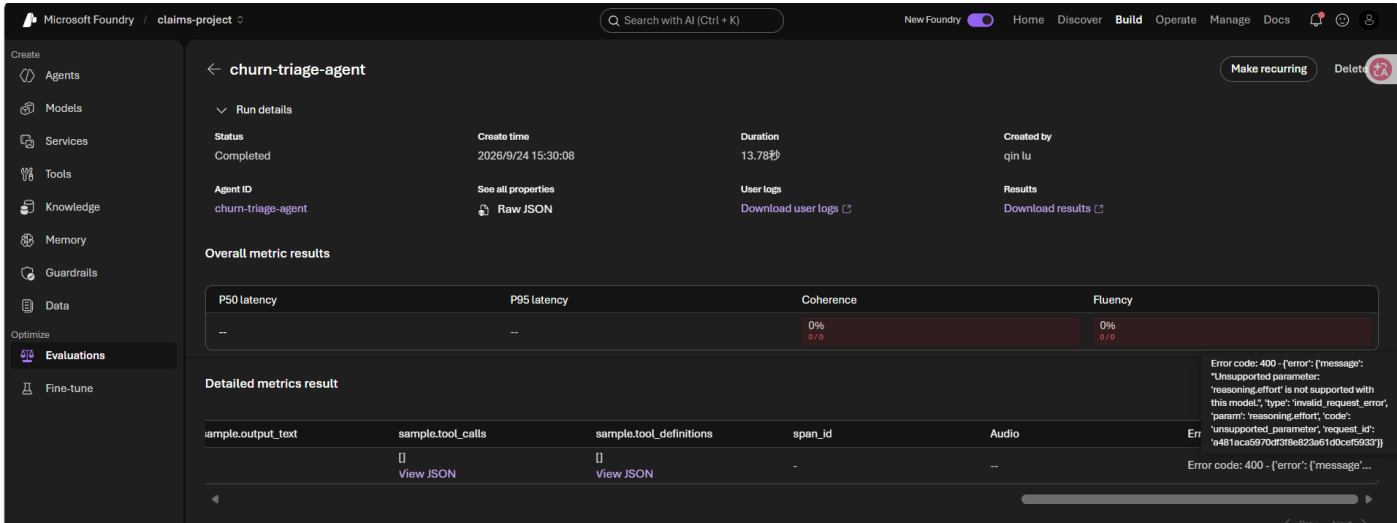
The V1–V11 iteration cycle led to three key conclusions:

- (1) **The root cause was not parallelism or throughput.**
The controlled single-record test in V7 eliminated the hypothesis that the failures were caused by dataset size, concurrency, or incomplete data ingestion.
- (2) **Agent design and model capability must be considered together.**
The experiments demonstrated that evaluation performance depended not only on the quality of the agent design and prompt, but also on the selected model's ability to reliably reason over the information, constraints, and decision logic provided by the query. A well-designed agent cannot compensate for insufficient model reasoning capability; therefore, agent architecture and model capability must be evaluated as an integrated system.
- (3) **Model configuration had a significant impact on multi-agent evaluation reliability.**
Changing both the model and its reasoning configuration ultimately resolved the reasoning failure. In particular, GPT-6-Astra with high reasoning effort succeeded where the tested GPT-6-Luna configurations did not.

The debugging process therefore evolved from data/input hypotheses → prompt and ground-truth validation → execution-level hypotheses → controlled single-record testing → model-level investigation.

This demonstrates the value of combining Online Evaluation with detailed Traces: evaluation identifies that the workflow is failing, while execution traces provide the evidence required to determine *where and why* the failure occurs.

Figure 4. Evaluation detailed metrics result



5. Workflow and Deployment

5.1 Python SDK Deployment — Baseline

The initial baseline workflow was deployed using Deploy.py with the Python SDK. The baseline consisted of two

agents and was used to validate the core agent workflow and evaluation approach.

The initial results were satisfactory, providing a stable foundation for progressively introducing additional agents and MCP integration.

Key Implementation Considerations

(1) Ground-truth fields should not be included in preview inputs

When previewing the workflow, the input data should contain only the features required by the workflow and should not include fields such as the expected churn risk level.

The workflow is responsible for generating the output of Agent 1, which is then passed downstream to Agent (2). Including the expected risk level in the input would introduce label leakage and invalidate the evaluation of the agent's reasoning capability.

(2) Independent Agent execution vs. true workflow orchestration

An important implementation distinction was identified when running the agents directly through `agents.py`. Although the standalone execution produced seemingly correct results, the two agents were actually executed independently. In the intended architecture, Agent 2 should consume and reason over the output generated by Agent 1. However, in the standalone script, Agent 2 generated the recommended action from its own LLM reasoning rather than receiving the actual upstream Agent 1 output.

This distinction is important because: Correct individual agent outputs do not necessarily demonstrate correct multi-agent orchestration. A true workflow must explicitly establish the data dependency and handoff between agents.

5.2 Foundry Workflow Deployment — MCP-Enabled Version

After validating the baseline, the workflow was extended to incorporate the remaining agents and an MCP-enabled action-execution layer.

The original plan was to integrate an Outlook MCP to automatically send the generated retention email. Due to the limited implementation timeframe, the Outlook integration was replaced with the Azure REST API Specifications MCP for end-to-end workflow validation.

The final workflow was created and deployed directly through Azure AI Foundry Workflow.

Compared with implementing the orchestration entirely through the Python SDK, the Foundry Workflow approach provided a more efficient development experience for this stage because the agent dependencies, workflow structure, and MCP integration could be configured and tested directly within the workflow environment.

The resulting implementation successfully validated the complete workflow:

Customer Data → Churn Triage → Retention Decision → Email Generation → MCP Tool Interaction

This deployment also provided a more faithful representation of the intended production architecture, where downstream agents explicitly consume upstream outputs rather than operating as isolated LLM calls.

5.3 GitHub Repository

The complete implementation, source code, technical documentation, evaluation details, and supporting screenshots are available in the project repository:

[Multi-Agent Customer Retention Platform — GitHub Repository](#)

The repository provides additional implementation details and supporting materials beyond the scope of this document, including the workflow configuration, agent implementation, evaluation iterations, and development screenshots.